

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Коммивояжёр. Вариант 1.

Студент гр. 4345

Лунченко Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы:

Изучить Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД).

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n - количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Пример входных данных

2

-1 18.97 22.36 19.42 3.61

18.97 -1 35.61 38.01 17.0
22.36 35.61 -1 16.28 21.19
19.42 38.01 16.28 -1 21.02
3.61 17.0 21.19 21.02 -1

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

91.92

2 3 0 4 1 2.

Выполнение работы

Базовое представление данных:

Матрица расстояний представляется как двумерный массив размера $N \times N$, где элемент $M[i][j]$ - стоимость перемещения из города i в город j . Для хранения и обработки матрицы используется библиотека numpy, что позволяет эффективно выполнять операции поиска минимумов по строкам и столбцам, а также операции удаления строк и столбцов.

Реализованные структуры данных:

Класс Node: представляет узел дерева ветвления

matrix - текущая приведённая матрица стоимостей

str_num, col_num - сопоставление индексов матрицы с номерами городов

d - нижняя граница стоимости для данного узла

edges_included, edges_excluded - множества включённых и исключённых дуг

left, right - указатели на левое (включение дуги) и правое (исключение дуги) поддеревья

Основные алгоритмы

Приведение матрицы (privod)

Выполняется редукция строк и столбцов: из каждой строки вычитается минимальный элемент, затем из каждого столбца. Сумма вычтенных значений образует нижнюю границу стоимости решения.

Временная сложность: $O(N^2)$

Выбор дуги для ветвления (find_heavy)

Для каждого нулевого элемента матрицы вычисляется «степень сожаления» (regret) - сумма минимумов строки и столбца без учёта самого элемента. Выбирается дуга с максимальным значением сожаления (самый «тяжёлый» ноль).

Временная сложность: $O(N^3)$

Включение дуги (left_tree)

При включении дуги (i, j) выполняются следующие действия:

Вычёркивание i -й строки и j -го столбца из матрицы

Добавление дуги в список включённых

Запрет обратной дуги (j, i)

Запрет дуги, замыкающей преждевременный цикл (конец цепочки $\rightarrow$ начало цепочки)

Повторное приведение матрицы

Временная сложность: $O(N^2)$

Исключение дуги (`right_tree`)

При исключении дуги (i, j) соответствующий элемент матрицы заменяется на бесконечность, после чего выполняется повторное приведение.

Временная сложность: $O(N^2)$

Проверка допустимости (`proverka`)

Проверяет, что в каждой строке и каждом столбце матрицы есть хотя бы один конечный элемент (не бесконечность). Если условие нарушено, узел считается неперспективным.

Временная сложность: $O(N^2)$

Отслеживание цепочек (`find_chain_end`, `find_chain_start`)

Проходят по всем включённым дугам и находят реальные начало и конец формирующейся цепочки городов для предотвращения преждевременного замыкания цикла.

Временная сложность: $O(K)$, где K - количество включённых дуг

Пошаговое описание работы

Инициализация:

Чтение матрицы стоимостей

Замена нулевых элементов (отсутствие прямого пути) на бесконечность

Создание корневого узла дерева ветвления

Начальное приведение матрицы для получения первой нижней границы

Рекурсивный поиск (функция `tree`):

Проверка: если все N дуг включены - найдено полное решение, сравнивается с текущим лучшим

Отсечение: если нижняя граница узла не меньше текущего рекорда - ветвь отбрасывается

Проверка допустимости: если в какой-либо строке или столбце все элементы бесконечны - ветвь отбрасывается

Выбор дуги для ветвления (самый «тяжёлый» ноль)

Построение левого (включение дуги) и правого (исключение дуги) поддеревьев

Рекурсивный обход в порядке: сначала более перспективная ветвь (с меньшей нижней границей), затем, при необходимости, менее перспективная

Восстановление маршрута (build_route, normalize_route):

По списку включённых дуг строится словарь переходов между городами

Определяется начальный город (тот, из которого выходят, но в который не входят)

Строится циклический маршрут обхода всех городов

Маршрут нормализуется так, чтобы начинаться с города 0

Эвристики и оптимизации:

Выбор самого «тяжёлого» нуля: позволяет максимально увеличить нижнюю границу в правой ветви и ускорить отсечение

Отсечение по нижней границе: ветви с нижней границей не меньше текущего рекорда не рассматриваются

Приоритетный обход: сначала исследуется ветвь с меньшей нижней границей (более перспективная)

Отслеживание цепочек: предотвращает замыкание преждевременных циклов, что соответствует классическому алгоритму метода ветвей и границ для задачи коммивояжёра

Оценка сложности алгоритма

Метод ветвей и границ для задачи коммивояжёра имеет экспоненциальную сложность в худшем случае - $O(2^{(N^2)})$. Однако благодаря эффективным отсечениям и эвристикам на практике средняя сложность значительно ниже.

Сложность по памяти: $O(N^2)$ для хранения матрицы на каждом уровне рекурсии. Глубина рекурсии не превышает N^2 , так как на каждом шаге размер матрицы уменьшается на 1 (в левой ветви) или заменяется один элемент на бесконечность (в правой ветви).

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования алгоритма Литтла

№	Входные данные	Выходные данные	Комментарий
1	3 0 1 3 3 0 1 1 2 0	3 0 1 2 0	Простой полный граф из 3 городов, проверка базовой работы
2	5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	78 0 4 2 3 1 0	Полный граф из 5 городов, оптимальный маршрут через все вершины
3	3 0 1 100 100 0 100 1 1 0	102 0 1 2 0	Асимметричный граф, выбор между маршрутами 102 и 201
4	3 0 5 0 5 0 0 0 0 0	no path	Граф с изолированной вершиной, путь невозможен
5	4 0 1 0 0 0 0 1 0 0 0 0 1 1 0 0 0	4 0 1 2 3 0	Единственный возможный путь, проверка на замыкание цикла

Постановка задачи

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Пример входных данных

```
2
-1 18.97 22.36 19.42 3.61
18.97 -1 35.61 38.01 17.0
22.36 35.61 -1 16.28 21.19
19.42 38.01 16.28 -1 21.02
3.61 17.0 21.19 21.02 -1
```

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

```
91.92
2 3 0 4 1 2
```

Выполнение работы

Базовое представление данных

Матрица расстояний представляется как двумерный массив размера $N \times N$, где элемент $M[i][j]$ - стоимость перемещения из города i в город j . Значение -1 обозначает отсутствие прямого пути между городами. Для хранения матрицы используется список списков.

Реализованные структуры данных

Основные структуры данных: `matrix` - матрица весов графа размером $N \times N$, `in_mst` - булев массив, отмечающий вершины, включённые в минимальное остовное дерево, `min_edge` - массив минимальных расстояний от каждой вершины до текущего MST, `parent` - массив родителей вершин в MST, `mst_adj` - список смежности для хранения рёбер MST, `euler_tour` - список для хранения эйлерова обхода MST, `order` - итоговый порядок посещения городов.

Основные алгоритмы

Построение минимального остовного дерева (алгоритм Прима): Алгоритм строит MST, начиная с заданной начальной вершины, на каждом шаге выбирается вершина с минимальным расстоянием до уже построенного дерева, значения -1 в матрице игнорируются при поиске минимума. Временная сложность: $O(N^2)$.

Построение списка смежности MST: По массиву родителей строится список смежности, содержащий пары (соседняя вершина, вес ребра), для каждой вершины сохраняются все инцидентные ей рёбра MST. Временная сложность: $O(N)$.

Эйлеров обход MST (DFS): Выполняется обход минимального остовного дерева в глубину, начиная с заданной стартовой вершины, при обходе соседние вершины посещаются в порядке возрастания веса ребра, что позволяет получить более короткий путь, при возврате из рекурсии вершина добавляется повторно, формируя эйлеров тур. Временная сложность: $O(N \log N)$ из-за сортировки рёбер.

Построение гамильтонова цикла: Из эйлера тура удаляются повторяющиеся вершины путём сохранения только первого вхождения каждой вершины, в конец маршрута добавляется стартовая вершина для замыкания цикла. Временная сложность: $O(N)$.

Вычисление длины маршрута: Для полученного порядка посещения городов вычисляется суммарная длина пути как сумма весов рёбер между последовательными вершинами маршрута. Временная сложность: $O(N)$.

Пошаговое описание работы

Инициализация: Чтение номера начальной вершины, чтение матрицы весов графа, инициализация массивов для алгоритма Прима.

Построение MST: Установка расстояния до начальной вершины равным 0, затем N раз выполняется выбор ближайшей невключённой вершины, добавление выбранной вершины в MST и обновление расстояний до соседних вершин.

Построение списка смежности: Для каждой вершины, имеющей родителя в MST, добавляется ребро в список смежности, рёбра добавляются в обоих направлениях.

DFS обход MST: Рекурсивный обход, начиная со стартовой вершины, посещение соседей в порядке возрастания веса ребра, добавление вершины в эйлеров тур при входе и при возврате.

Построение маршрута: Из эйлера тура удаляются повторные вхождения вершин, в конец добавляется стартовая вершина для замыкания цикла.

Вывод результата: Вычисление и вывод длины полученного маршрута с точностью до двух знаков, вывод последовательности вершин маршрута.

Эвристики и оптимизации

Сортировка рёбер при DFS: Посещение соседей в порядке возрастания веса ребра позволяет получить более короткий гамильтонов цикл по сравнению со случайным порядком обхода. Выбор начальной вершины: Алгоритм учитывает заданную начальную вершину, что позволяет получить

маршрут, начинающийся и заканчивающийся в требуемом городе. Игнорирование отсутствующих рёбер: Значения -1 в матрице весов корректно обрабатываются как отсутствие прямого пути.

Оценка сложности алгоритма

Алгоритм Прима имеет временную сложность $O(N^2)$ для полного графа, DFS обход MST выполняется за $O(N \log N)$ из-за сортировки рёбер, общая временная сложность алгоритма составляет $O(N^2)$. Сложность по памяти: $O(N)$ для хранения массивов алгоритма Прима и списка смежности MST. Алгоритм является 2-приближенным, что означает, что длина найденного маршрута не более чем в 2 раза превышает длину оптимального маршрута для метрической задачи коммивояжёра.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 2.

Таблица 2 – Результаты тестирования алгоритма

№	Входные данные	Выходные данные	Комментарий
1	0 -1 1 3 1 -1 1 3 1 -1	5.00 0 1 2 0	Простой полный граф из 3 городов
2	0 -1 -1 -1 -1 -1 -1 -1 -1 -1	no path	Все вершины изолированы, путь невозможен
3	2 -1 1 13 23 7 12 -1 15 18 28 21 29 -1 33 28 23 19 34 -1 38 5 40 7 39 -1	85.00 2 0 1 3 4 2	Полный граф из 5 городов со стартом из 2

№	Входные данные	Выходные данные	Комментарий
4	0 -1 5 -1 5 -1 -1 -1 -1 -1	10.00 0 1 0	Граф с изолированной вершиной
5	0 -1 1 -1 -1 -1 -1 1 -1 -1 -1 -1 1 1 -1 -1 -1	4.00 0 1 2 3 0	Единственный возможный путь

Выводы

В ходе выполнения работы были реализованы два алгоритма для решения задачи коммивояжёра: точный метод ветвей и границ (алгоритм Литтла) и приближённый 2-приближенный алгоритм на основе минимального остовного дерева. Алгоритм Литтла реализует поиск оптимального маршрута путём построения дерева решений с приведением матрицы стоимостей, выбором самого тяжёлого нуля для ветвления и отсечением по нижней границе. Приближённый алгоритм строит минимальное остовное дерево алгоритмом Прима, выполняет его эйлеров обход и формирует гамильтонов цикл путём удаления повторяющихся вершин, гарантируя двукратное приближение к оптимуму для метрической задачи.

Несмотря на экспоненциальную сложность алгоритма Литтла в худшем случае, применение эвристик выбора тяжёлого нуля, отслеживания цепочек для предотвращения подциклов и приоритетного обхода ветвей позволяет находить точное решение для графов размером до 15 вершин за приемлемое время. Приближённый алгоритм с полиномиальной сложностью $O(N^2)$ эффективен для графов большего размера, когда достаточно получить решение, близкое к оптимальному.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл little.py

```
import math
import numpy as np

class Node:
    def __init__(self, matrix, d, edges_included, edges_excluded,
                  parent, str_num, col_num):
        self.matrix = matrix.copy()
        self.str_num = str_num.copy()
        self.col_num = col_num.copy()
        self.d = d
        self.edges_included = edges_included.copy()
        self.edges_excluded = edges_excluded.copy()
        self.N = len(matrix)
        self.parent = parent
        self.left = None
        self.right = None

    def find_d(self, i, j):
        original = self.matrix[i][j]
        self.matrix[i][j] = math.inf
        d = np.min(self.matrix[i]) + np.min(self.matrix[:, j])
        self.matrix[i][j] = original
        return d

    def find_heavy(self):
        d = -math.inf
        I, J = 0, 0
        for i in range(self.N):
            for j in range(self.N):
                if self.matrix[i][j] == 0:
                    regret = self.find_d(i, j)
                    if d < regret:
                        d = regret
```

```

        I, J = i, j

    return I, J

def privod(self):
    d = 0
    for i in range(self.N):
        m = np.min(self.matrix[i])
        if not math.isinf(m):
            d += m
            self.matrix[i] -= m
    for j in range(self.N):
        m = np.min(self.matrix[:, j])
        if not math.isinf(m) and m > 0:
            d += m
            self.matrix[:, j] -= m
    return d

def find_chain_end(self, start_city):
    current = start_city
    while True:
        found = False
        for edge in self.edges_included:
            if edge[0] == current:
                current = edge[1]
                found = True
                break
        if not found:
            break
    return current

def find_chain_start(self, end_city):
    current = end_city
    while True:
        found = False
        for edge in self.edges_included:
            if edge[1] == current:
                current = edge[0]
                found = True
                break

```



```

        if not found:
            break
    return current

def left_tree(self, i, j):
    new_matrix = self.matrix.copy()

    from_city = self.str_num[i]
    to_city = self.col_num[j]
    chain_end = self.find_chain_end(to_city)
    chain_start = self.find_chain_start(from_city)

    if chain_end in self.str_num and chain_start in self.col_num:
        r = self.str_num.index(chain_end)
        c = self.col_num.index(chain_start)
        new_matrix[r][c] = math.inf

    new_matrix = np.delete(new_matrix, j, axis=1)
    new_matrix = np.delete(new_matrix, i, axis=0)

    new_str_num = self.str_num.copy()
    new_col_num = self.col_num.copy()

    new_str_num.pop(i)
    new_col_num.pop(j)

    new_node = Node(new_matrix, self.d,
                    self.edges_included.copy(),
self.edges_excluded.copy(),
                    self, new_str_num, new_col_num)

    new_node.edges_included.append([from_city, to_city])
    if chain_end != chain_start:
        new_node.edges_excluded.append([chain_end, chain_start])
    else:
        new_node.edges_excluded.append([to_city, from_city])
    new_node.d += new_node.privod()
    self.left = new_node

```

```

def right_tree(self, i, j):
    new_matrix = self.matrix.copy()
    new_matrix[i][j] = math.inf
    new_node = Node(new_matrix, self.d,
                    self.edges_included.copy(),
self.edges_excluded.copy(),
                    self, self.str_num, self.col_num)
    new_node.edges_excluded.append([self.str_num[i],
self.col_num[j]])
    new_node.d += new_node.privod()
    self.right = new_node

def proverka(self):
    if self.N == 1:
        return 1
    for i in range(self.N):
        if np.min(self.matrix[i]) == math.inf:
            return 0
    for j in range(self.N):
        if np.min(self.matrix[:, j]) == math.inf:
            return 0
    return 1

def tree(node, N, best_path, best):
    if node is None:
        return best_path, best

    if len(node.edges_included) == N:
        if node.d < best:
            return node.edges_included.copy(), node.d
        return best_path, best

    if node.d >= best:
        return best_path, best

    if node.proverka() == 0:
        return best_path, best

```

```

i, j = node.find_heavy()
node.left_tree(i, j)
node.right_tree(i, j)

if node.left and node.right:
    if node.left.d <= node.right.d:
        best_path, best = tree(node.left, N, best_path, best)
        if node.right.d < best:
            best_path, best = tree(node.right, N, best_path, best)
    else:
        best_path, best = tree(node.right, N, best_path, best)
        if node.left.d < best:
            best_path, best = tree(node.left, N, best_path, best)
elif node.left:
    best_path, best = tree(node.left, N, best_path, best)
elif node.right:
    best_path, best = tree(node.right, N, best_path, best)

return best_path, best

```

```

def build_route(edges):
    if not edges:
        return []
    nxt = {edge[0]: edge[1] for edge in edges}
    from_cities = set(nxt.keys())
    to_cities = set(nxt.values())
    start_candidates = from_cities - to_cities
    start = start_candidates.pop() if start_candidates else edges[0][0]

    route = [start]
    current = start
    while len(route) <= len(edges):
        current = nxt[current]
        route.append(current)
        if current == start:
            break
    return route

```

```

def normalize_route(route):
    route_zero = [x - 1 for x in route]
    if 0 in route_zero:
        idx = route_zero.index(0)
        route_zero = route_zero[idx:] + route_zero[1:idx + 1]
    return route_zero

def main():
    N = int(input())
    m = []
    for _ in range(N):
        row = list(map(float, input().split()))
        m.append(row)
    m = np.array(m)
    m = np.where(m == 0, math.inf, m)

    str_num = [i + 1 for i in range(N)]
    col_num = [i + 1 for i in range(N)]
    root = Node(m, 0, [], [], None, str_num, col_num)
    root.d = root.privod()

    best_path, best = tree(root, N, [], math.inf)

    if math.isinf(best):
        print("no path")
    else:
        print(int(best))
        path = normalize_route(build_route(best_path))
        print(' '.join(str(city) for city in path))

if __name__ == "__main__":
    main()

```

Файл adomod.py

```

def solve():
    first = input().strip()

```

```

if not first:
    return
start = int(first)

rows = []
for i in range(100):
    line = input().strip()
    if not line:
        break
    rows.append(list(map(float, line.split())))

n = len(rows)
matrix = rows

INF = float('inf')
in_mst = [False] * n
min_edge = [INF] * n
parent = [-1] * n
min_edge[start] = 0

for i in range(n):
    u = -1
    best = INF
    for v in range(n):
        if not in_mst[v] and min_edge[v] < best:
            best = min_edge[v]
            u = v
    if u == -1:
        break
    in_mst[u] = True
    for v in range(n):
        w = matrix[u][v]
        if w != -1 and not in_mst[v] and w < min_edge[v]:
            min_edge[v] = w
            parent[v] = u

mst_adj = [[] for i in range(n)]
for v in range(n):
    if parent[v] != -1:

```

```

        p = parent[v]
        w = matrix[p][v]
        mst_adj[p].append((v, w))
        mst_adj[v].append((p, w))

visited = [False] * n
euler_tour = []

def dfs(u):
    visited[u] = True
    euler_tour.append(u)
    for v, w in sorted(mst_adj[u], key=lambda x: x[1]):
        if not visited[v]:
            dfs(v)
            euler_tour.append(u)

dfs(start)

order = []
seen = [False] * n
for v in euler_tour:
    if not seen[v]:
        seen[v] = True
        order.append(v)
order.append(start)

path_length = 0.0
for i in range(len(order) - 1):
    u = order[i]
    v = order[i + 1]
    path_length += matrix[u][v]

print(f"{path_length:.2f}")
print(" ".join(str(x) for x in order))

if __name__ == "__main__":
    solve()

```